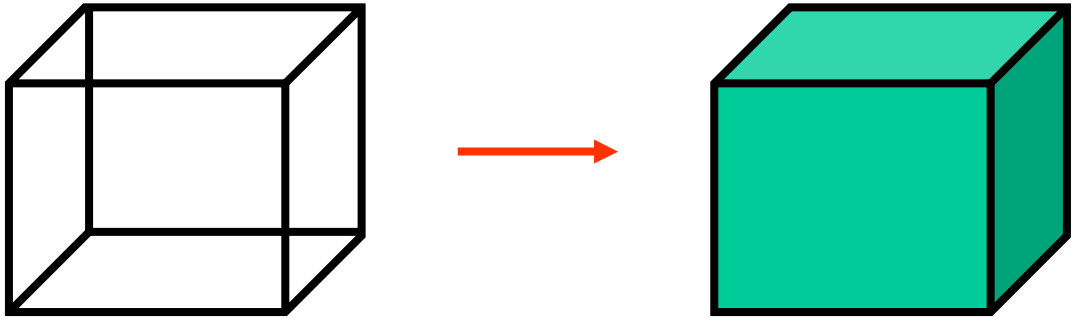
Visible-Surface Detection

Problem:

Given a scene and a projection,

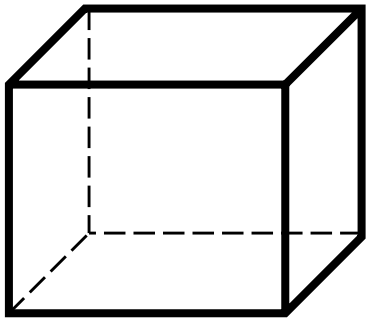
what can we see?



Visible-Surface Detection

Two main types of algorithms:

- *Object space: Determine which part of the object are visible*
- *Image space: Determine per pixel which point of an object is visible*



Object space

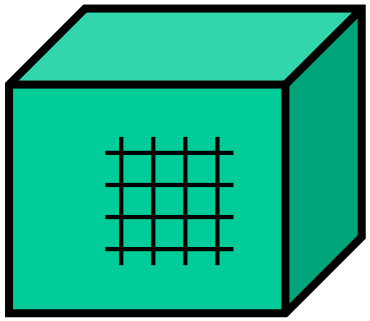


Image space

Visible-Surface Detection

Four algorithms:

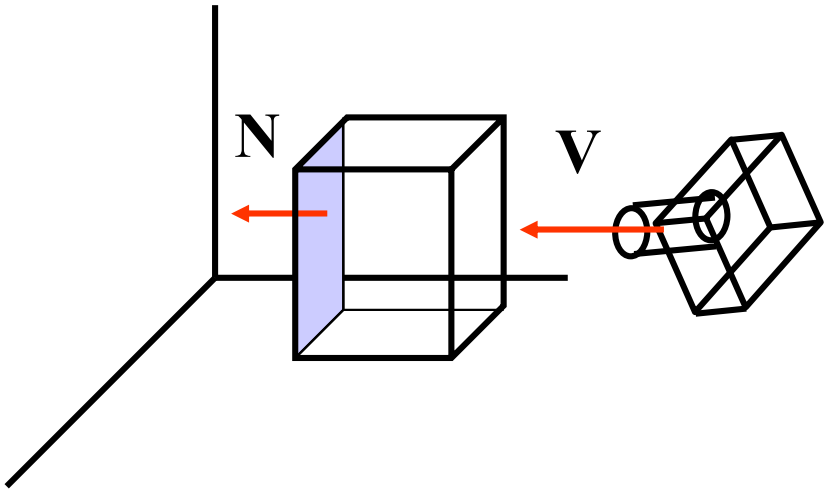
- Back-face elimination
- Depth-buffer
- Depth-sorting
- Ray-casting

But there are many other.

Back-face elimination 1

We cannot see the back-face of solid objects:

Hence, these can be ignored

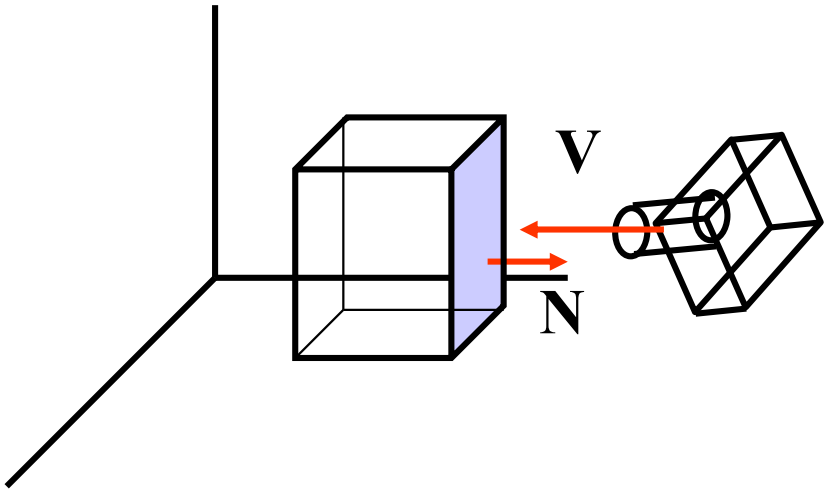


$$\mathbf{V} \cdot \mathbf{N} > 0 : \text{back face}$$

Back-face elimination 2

We cannot see the back-face of solid objects:

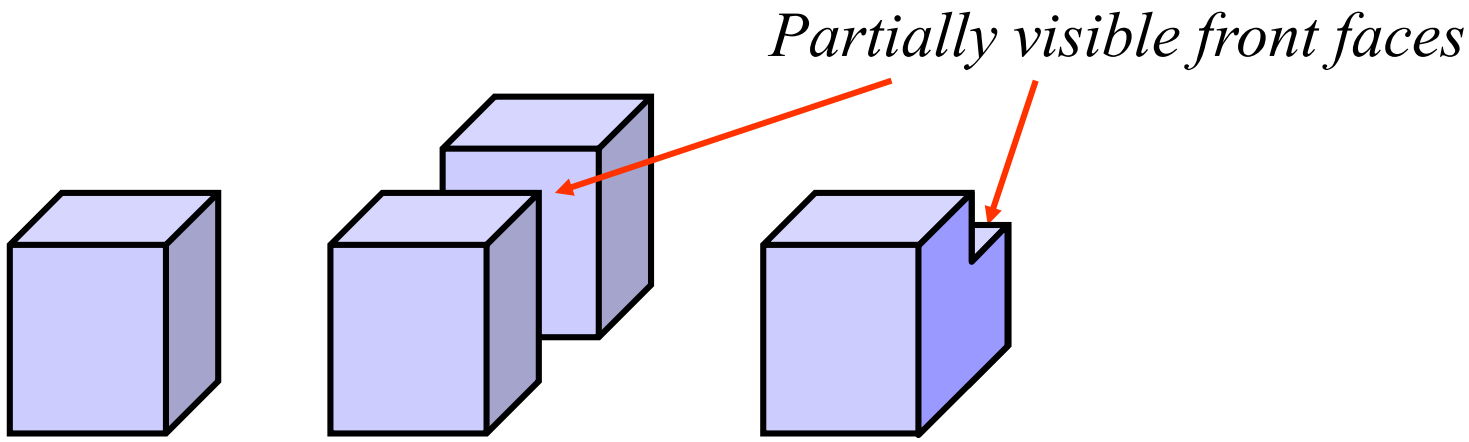
Hence, these can be ignored



$$\mathbf{V} \cdot \mathbf{N} < 0 : \text{front face}$$

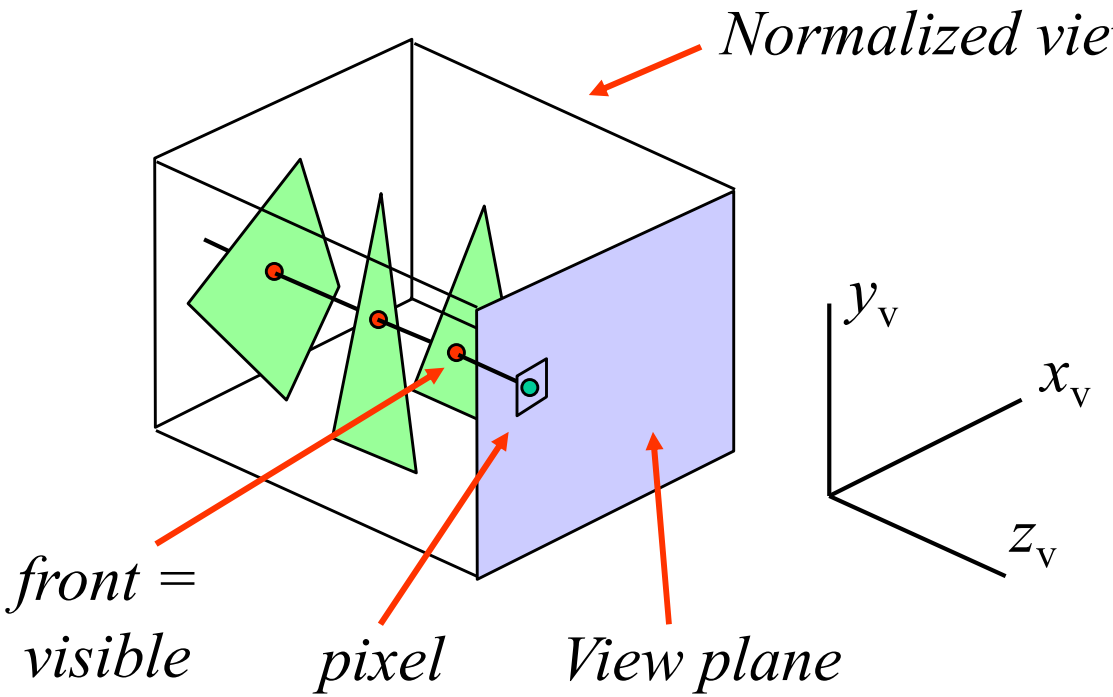
Back-face elimination 3

- Object-space method
- Works fine for convex polyhedra: ±50% removed
- Concave or overlapping polyhedra: require additional processing
- Interior of objects can not be viewed



Depth-Buffer Algorithm 1

- Image-space method
- Aka *z-buffer algorithm*



Algorithm:
Draw polygons,
Remember the color most in front.

Depth-Buffer Algorithm 2

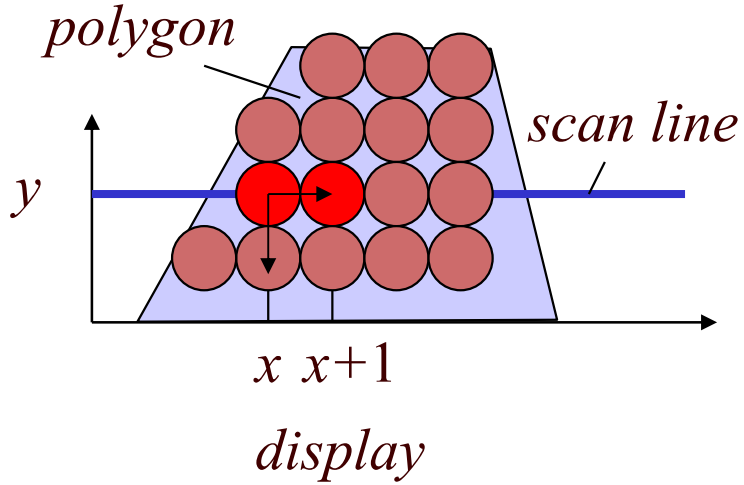
```
var zbuf: array[N,N] of real;          { z-buffer: 0=near, 1=far }
    fbuf: array[N,N] of color;         { frame-buffer }

For all 1<= i, j <=N do
        zbuf[i,j] := 1.0; col[i,j] := BackgroundColour;
For all polygons do                    { scan conversion }
    For all covered pixels (i,j) do
        Calculate depth z;
        If z < zbuf[i,j] then { closer! }
            zbuf[i,j] := z;
            fbuf[i,j] := surfacecolor(i,j);
```

Sorting

Depth-Buffer Algorithm 3

Fast calculation *z*: use coherence.



Plane : $Ax + By + Cz + D = 0$

Hence : $z(x, y) = \dfrac{-Ax - By - D}{C}$

Also : $z(x+1, y) = \dfrac{-A(x+1) - By - D}{C}$

Thus : $z(x+1, y) = z(x, y) - \dfrac{A}{C}$

Also : $z(x, y) = z(x, y-1) + \dfrac{B}{C}$

Depth-Buffer Algorithm 4

+ Easy to implement
+ Hardware supported
+ Polygons can be processed in arbitrary order
+ Fast: ~ #polygons, #covered pixels

- Costs memory
- Color calculation sometimes done multiple times
- Transparancy is tricky

END